Players’ Ball Possession Estimation During Football Matches

Davide Perniconi - 1889270
Stefano Passanante - 2158181
Robert Cristian Iacobus - 1834884

Computer Vision, a.y. 25/26
Sapienza University of Rome

Motivation and problem statement

Problem: ball possession is a fundamental statistic in soccer analytics, used to assess:

- Team dominance
- Tactical behavior
- Player involvement and performance

Manual annotation is:

- Time-consuming
- Subjective
- Not scalable

Goal: automatically compute player-level and team-level ball possession from broadcast soccer videos using Computer Vision techniques

Key challenges:

- Small and fast-moving ball
- Motion blur
- Frequent occlusions
- Similar player appearance
- Low/medium video resolution (≈ 640 px)

Output example

Related work and inspiration

Our work is inspired by recent advances in soccer video tracking, particularly from the **SoccerNet** series:

- *SoccerNet-Tracking (Cioppa et al., CVPR 2022)*
  - Provides a fully annotated MOT dataset
  - Evaluates different MOT trackers on their newly introduced dataset
- *SoccerNet Game State Reconstruction (Somers et al., CVPRW 2024)*
  - Secondary inspiration for pipeline design and modular tracking + ReID ideas
  - We do not implement game state reconstruction

We leverage these ideas to design a ball possession estimation pipeline

SoccerNet-Tracking (2022)

*"In practice, **several metrics typically supports the scouting choices**. For instance, player endurance along a full game, total distance run, top speed in counter-attacks, **number of ball possessions**, assists or goals are only few examples of characteristics that your team would consider before hiring your next soccer talent."*

- **Dataset** comprises actions from 12 soccer games (2019 Swiss Super League) captured in Full-Hd and at 25 fps (MOT20 format)
- **Baseline evaluation** → around 70% HOTA

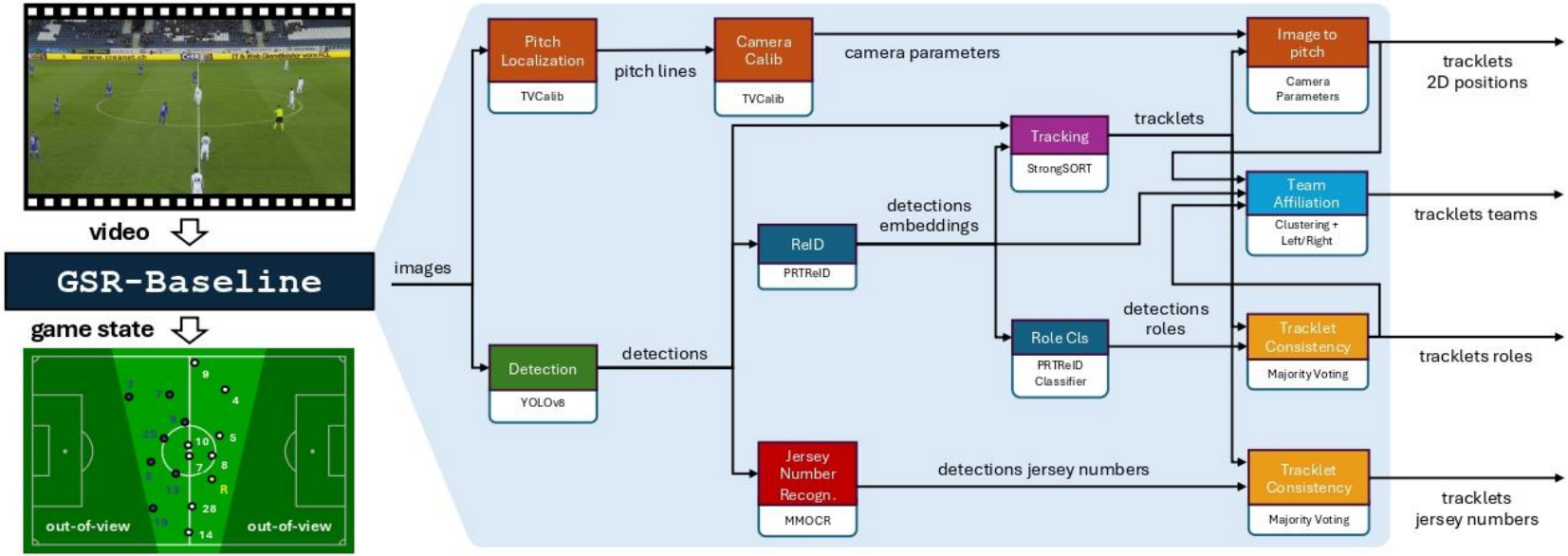
SoccerNet Game State Reconstruction (2024)

The work introduces the Game State Reconstruction Task (GSR) and a dataset for it. We are interested in the proposed baseline pipeline:

- Pre-trained **YOLOv8** model as athlete detector (keep only *person* class)
- **StrongSORT** as MOT, taking, besides spatio-temporal informations, appearances cues by PRTreID
- **PRTreID**, tracker + multi-task model designed for sports
- **MMOCR** for jersey number recognition

"experiments with our proposed baseline reveal the inherent complexity of the GSR task and the significant interdependencies among its various subtasks"

SoccerNet Game State Reconstruction (2024)



Our project pipeline overview

1. Object Detection (YOLO fine-tuned)
2. Multi-Object Tracking (BoT-SORT)
3. Re-Identification (OSNet)
4. Refinement techniques: ~~ball interpolation~~, hungarian algorithm, swap guard.
5. Possession statistics computation

YOLO: You Only Look Once

Single-stage object detector:

- Entire image processed in one forward pass
- Direct prediction of bounding boxes, objectness score and class probabilities

Core components:

- Backbone (feature extraction)
- Head (bounding box regression + classification)

YOLO fine-tuning

Domain adaptation to soccer with a fine-tuning phase

- **YOLO11m** as base model, pre-trained on COCO dataset
- SoccerNet Tracking 2022 train split further divided → 85% train and 15% validation
- New classes: { *ball, gk_left, gk_right, player_left, player_right, referee* }
- Conversion from MOT to YOLO detection format
- Resolution fixed at 640x640px
- Optimized confidence threshold based on **F1-Score**

We leverage pretrained general features while specializing on soccer-specific objects

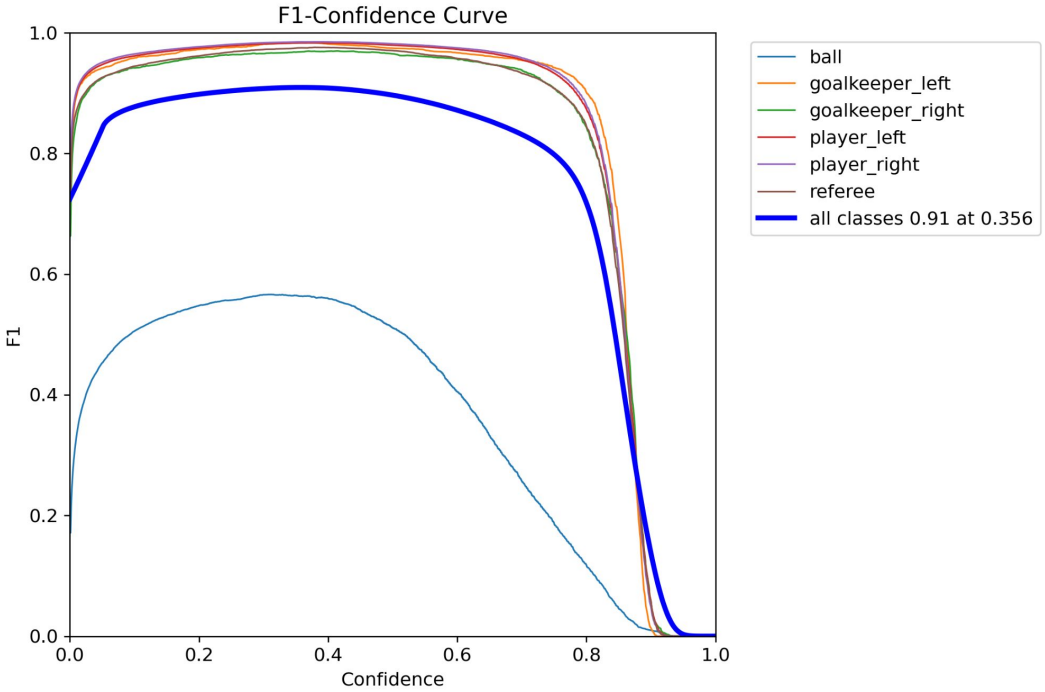
YOLO fine-tuning

Domain adaptation to soccer with a fine-tuning phase but we had problems:

- Frequent Kaggle session timeouts/crashes after 12 hours even with moderate VRAM consumption
- We reached 22 epochs with Kaggle GPU T4 x2
- Impossible to train on large amount of data in SoccerNet dataset for >30 epochs

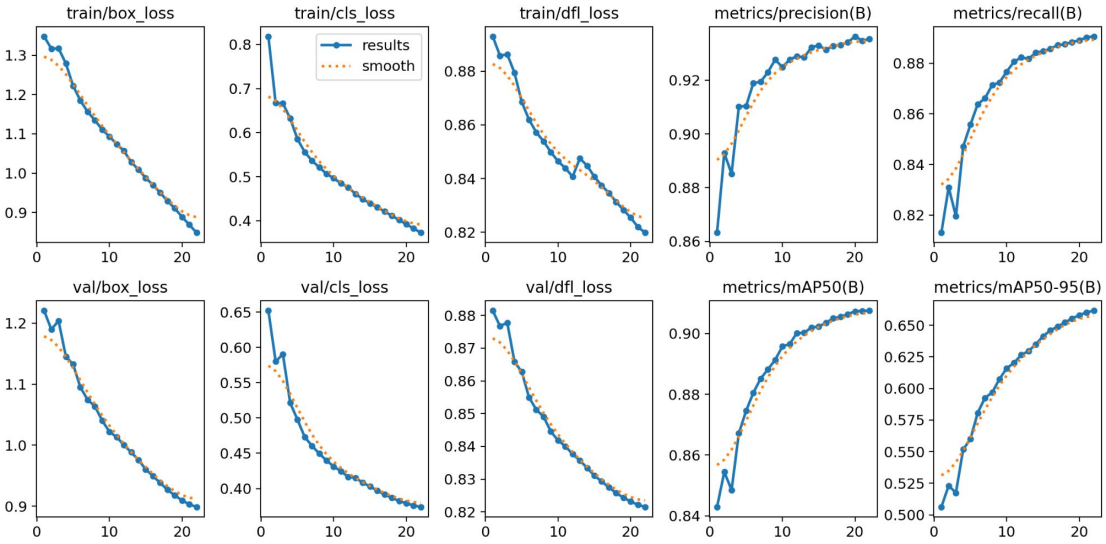
YOLO fine-tuning: F1-confidence curve

- Trade-off between the model's Precision and Recall across varying confidence thresholds
- Peak global F1 score of 0.91 achieved at confidence **threshold = 0.356**
- The ball class represents a significant bottleneck in the model's overall performance



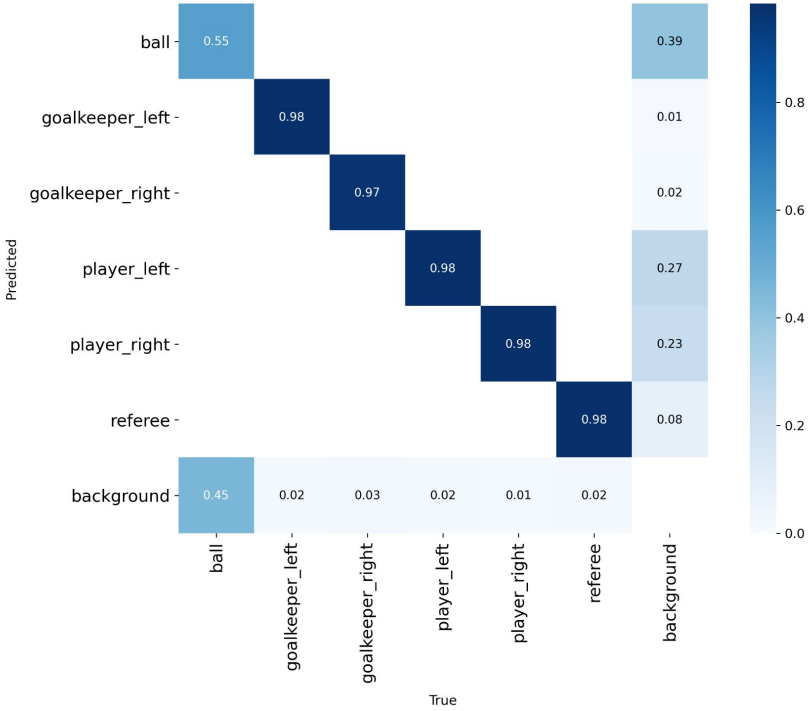
YOLO fine-tuning: loss & results

- Progressive convergence of loss metrics during fine-tuning
- Strong performance metrics achieved with high precision and recall **for “big” subjects**
- More fine-tuning would probably improve ball performances (“small subject”)



YOLO fine-tuning: confusion matrix normalized

- Excellent classification performance for goalkeepers, players, and referee with 0.97-0.98 recall
- **Low inter-class confusion** between player types indicates strong feature learning



Multi-Object Tracking (BoT-SORT)

- Real-time multi-object tracker that **combines motion prediction and optional ReID** appearance features to maintain consistent IDs over frames
- (Should be) robust to short-term occlusions
- Easily integrable with YOLO → model.track()
- Main concepts (for all MOTs):
  - Track is the sequence of positions of a single object. Ideally covers all its presence duration in the video
  - Each object gets a unique ID that persists as long as it is tracked.
  - Tracklet is a short, continuous segment of a track / cover a short and continuous interval
- Usage of **OSNet** ReID embeddings

OSNet embeddings

- OSNet_x1_0 is a lightweight NN designed for person re-identification (ReID)
- **Omni-Scale** → captures both fine-grained local details and global context simultaneously
- Designed to handle variations in pose, illumination, and viewpoint robustly.
- Crop resizing: 256 x 128 and normalization (**Market-1501** protocol)
- Output is high-dimensional L2-normalized embedding
- Market-1501 contains 32k annotated images of 1,501 identities captured by six cameras (video surveillance)
- ResNet50 was also considered for feature extraction but OSNet_x1_0 was the preferred option due to its training on a realistic dataset and its better ability to capture details at lower resolution

Using BoT-SORT with OSNet_x1_0

Combine tracking (BoT-SORT) with ReID features (OSNet) to track objects across multiple frames

Procedure:

- Detection (in our case with YOLO)
- Extract ReID features for each detected object using OSNet_x1_0.
- Tracking & Association

Expected benefits:

- Embeddings help maintain identity when motion alone is ambiguous (e.g., occlusions, crossing paths).
- Continuous trajectories with consistent IDs for each individual across frames

BoT-SORT parameters

We empirically found params of our chosen tracker. Most important ones:

- Track buffer = # frames a tracklet can remain “alive” even if the object is temporarily lost
- Track high thresh = control which detected objects are trusted enough to enter or stay in the tracker
- Proximity_thresh & appearance_thresh = tune the balance between motion and appearance

ReID Memory Management Strategy

Problem: We hypothesize that using a single embedding per identity is insufficient, since player appearance varies significantly over time due to pose changes, viewpoint, scale and partial occlusions.

Our solution:

- Maintain a fixed-size **FIFO** queue of embeddings per ID (updates with recent infos)
- Add a new embedding only if the maximum cosine similarity with the stored embeddings is below a given threshold
- When the embedding queue is full for a given ID, the most redundant embedding is removed.

Expect benefit: more robustness to pose changes, lighting variations and partial occlusions and avoid redundant representations

Hungarian Algorithm

Problem: identity preservation requires associating existing tracklets with new detections at each time step; detectors provide frame-level detections without identity information → data association is required

- Association problem formulated as a bipartite matching task between existing tracklets (rows) and current-frame detections (columns) → solved with **Hungarian Algorithm**
- Each tracklet *i* stores a fixed-size set of OSNet embeddings $\mathcal{E}_i = \{\mathbf{e}_i^1, \mathbf{e}_i^2, \ldots, \mathbf{e}_i^K\}$
- Each detection *j* in the current frame is represented by a <u>single</u> embedding d_j
- Association cost between *i* and *j* is min cosine distance $c(i,j) = \min_{k \in \{1,\ldots,K\}} \mathrm{cosine_dist}(\mathbf{e}_i^k, \mathbf{d}_j)$
- Cost matrix is computed **at every frame** and the algorithm returns the optimal 1-to-1 matching
- If similarity >= threshold configured assign existing ID, otherwise -> keep YOLO ID

Expect benefit: reduced ID switches and better track continuity

Occlusion handling and swap prevention

Problem: identity switches frequently occur during close interactions (duels, tackles), dregrating tracking performances (IDF1, ID switches)

Our solution tries to prevent accidental ID swaps while preserving tracking continuity

1. Group detections with IoU higher than a threshold
2. For each detection/cluster, it identifies the most overlapping box from the previous frame
3. A potential swap is flagged if the current ID differs from the historical ID at that spatial location
4. When a conflict is detected within a cluster, the system decides whether to accept the swap or revert it based on Re-ID Confidence.

Expected benefit: improved ID stability and reduced ID switches during close player interactions

Ball interpolation

Problem: soccer ball is very small relative to frame size, subject to strong motion blur (continuous fast movement and often partially or fully occluded by players

Our (tentative) solution is to perform ball interpolation in order to fill the gaps in a trajectory:
For each frame, if the ball is not detected, its position is estimated using linear or motion-model-based interpolation

In our experiments, interpolation **did not improve** tracking performance.
Likely reason: the ball detector often fails to produce detections, leaving too few reliable anchor points for interpolation → interpolated positions in these cases can be noisy or misleading, reducing trajectory accuracy rather than improving it.

Ball possession estimation

Definition: a player is in possession if ball bounding box overlaps for a minimum temporal duration

Calculated metrics:

- Player possession time (seconds)
- Team possession percentage

Evaluation protocol

- Reserved **test split** from Player Tracking 2022 (MOT20 format)
- We assess both detection quality and tracking performance for players and ball
- For each video sequence (frame-by-frame):
  - Compare tracker predictions with ground truth annotations (e.g., IoU > 0.5)
  - Aggregate metrics across all sequences to get overall performance
- Ball possession calculated stats **cannot be directly compared** (no gt in dataset). We assume that their reliability comes from the goodness of (especially ball) tracking
- Our main metric is **High Order Tracking Accuracy (HOTA)**, decomposes tracking quality into detection accuracy and association accuracy, and combines them into a single interpretable score

Evaluation metrics

- Recall: fraction of ground-truth objects successfully detected
- Precision: fraction of predicted objects that are correct
- False Positives: # incorrect detections.
- Misses: # ground-truth objects not detected
- **HOTA**: most considered metric in research
- **MOTA**: combines misses, false positives, and ID switches into overall tracking accuracy. MOTA value can be dominated by detection performance
- MOTP: average spatial accuracy of matched objects (how well bounding boxes align).
- IDF1: F1 score for correct identity matching over time (stability over time)
- ID Switches: Number of times a tracked ID changes incorrectly

$$\mathrm{HOTA} = \sqrt{\mathrm{DetA} \times \mathrm{AssA}} \qquad \mathrm{MOTA} = 1 - \frac{\sum_t (\mathrm{FN}_t + \mathrm{FP}_t + \mathrm{IDSW}_t)}{\sum_t \mathrm{GT}_t}$$

Results interpretation

- We used motmetrics instead of TrackEval framework, due to the poor extendibility of the last one
- Our systems suffers in ball detection → limited fine-tuned influence poor tracking performance
- **Important consideration**:
  - Each video in the SN dataset showcases significant 30-second sequences characterised by high dynamism, collisions and potential goals → **always high intensity**
  - This makes tracking more challenging, as many real-game moments are 'steady' → **intensity varies**

Results

- Strong detection performance, with a recall of 81.3% and precision of 88.1%
- Moderate overall tracking accuracy (MOTA score of 63.0%)
- Good spatial alignment between predicted and ground-truth object locations (MOTP of 0.189)
- Identity consistency is limited with many identity switches
- The solution demonstrates reliable object detection and localization, identity association is bottleneck

	Recall	Precision	FP	FN	MOTA	MOTP	IDF1	IDs
OVERALL	81.3%	88.1%	61944	105574	63.0%	0.189	40.7%	41442

	DETA	ASSA	HOTA
OVERALL	59.2%	24.7%	37.7%

Conclusions and observations

More fine-tuning is required, we were constrained by limited environment provided by Kaggle/Colab:

- Limited # epochs
- Scaled from Full-Hd to 640x640px

The **integration of different techniques** surely helped to obtain a more stable ID assignment over time:

- Hungarian algorithm for global assignment
- Multi-embedding representation of detection crops
- Occlusion swap guard

Soccer tracking remains an hard task:

- Ball is small and rapid
- Frequent occlusions
- Camera movements and replays

Conclusions and observations (2)

- Paradoxically, even the style of teams could influence outcomes (teams that tend to kick away the ball more frequently etc.)
- HOTA metric calculation very hard (TrackEval was not suitable, experimental support in motmetrics)
- Extensive “hypertuning” required on BOT-Sort and other trackers to achieve better HOTA very difficult on our Kaggle environment
- For a real-time scenario, a possible **multi-threaded approach** could be:
  - Detection and Osnet embeddings calculation **at batch level**
  - Tracking in parallel (but still frame-by-frame)
- We found this project extremely stimulating:
  - Practically understood the **key challenges** that arise for a production-grade soccer analysis system
  - **Literature review** of many possible solutions

References

- **Reference papers:**
  - Cioppa, Anthony, et al. "Soccernet-tracking: Multiple object tracking dataset and benchmark in soccer videos." Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2022
  - Somers, Vladimir, et al. "SoccerNet game state reconstruction: End-to-end athlete tracking and identification on a minimap." Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024
- **Tools:**
  - YOLO11 → Jocher, Glenn, Jing Qiu, and A. Chaurasia. "Ultralytics yolo11." gitHub repository (2024).
  - BoT-SORT → Aharon, Nir, Roy Orfaig, and Ben-Zion Bobrovsky. "BoT-SORT: Robust associations multi-pedestrian tracking." arXiv preprint arXiv:2206.14651 (2022)
  - OSnet → Zhou, Kaiyang, et al. "Omni-scale feature learning for person re-identification." Proceedings of the IEEE/CVF international conference on computer vision. 2019
  - HOTA → Luiten, Jonathon, et al. "Hota: A higher order metric for evaluating multi-object tracking." International journal of computer vision 129.2 (2021): 548-578
  - TrackEval → https://github.com/JonathonLuiten/TrackEval by Jonathon Luiten, Arne Hoffhues
  - ResNet → He, Kaiming, et al. "Deep residual learning for image recognition." Proceedings of the IEEE conference on computer vision and pattern recognition. 2016
- **Slides:**
  - https://github.com/pietro-nardelli/sapienza-ppt-template *[Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)]*

Thank you for the attention!